Neural Information Retrieval System

Yacine Dosso · March 2026

Stack: Python, NLTK, sentence-transformers, TensorFlow Hub

Dataset: SciFact - scientific claim verification corpus

1. Overview

This project builds a full Information Retrieval pipeline on the SciFact dataset, moving from a classical keyword-based system to neural reranking using pre-trained sentence embedding models. The goal was to understand how retrieval quality changes when you go from matching words to matching meaning.

The SciFact corpus is a collection of scientific abstracts paired with claims. Retrieving relevant documents in this setting is genuinely hard: a query might ask about "neural plasticity" while the relevant document talks about "synaptic adaptation." A keyword-based system fails here. A system that understands language does not.

This project also served as a direct bridge to my interest in Music Information Retrieval. The core pipeline; indexing, retrieval, and reranking, maps directly onto audio search problems. Instead of matching a query to scientific documents, the same architecture can match a query song to a corpus of audio tracks. Understanding how neural embeddings outperform keyword matching in text IR raises the same question for audio: can learned audio embeddings capture musical similarity better than handcrafted features like MFCCs?

2. Pipeline

corpus.jsonl + queries.jsonl
|
Preprocessing (tokenization, stopwords, stemming)
|
Inverted Index (TF per term per doc, DF, doc lengths)
|
TF-IDF Retrieval → Top-100 candidates per query
|
Neural Reranking (MPNet or USE)
|
Evaluation (MAP, P@10)

3. System Components

- preprocessing.py

Each document and query goes through the following steps:

- Lowercasing

- Tokenization using regular expressions (alphabetic tokens only)
- Stop word removal using the NLTK English stopword list
- Porter stemming to reduce words to their root form

This ensures consistent term matching between queries and documents, and reduces vocabulary noise. For example, "retrieving", "retrieved", and "retrieval" all map to the same stem.

- **indexing.py**

Builds an inverted index that maps each term to a dictionary of document IDs and their term frequencies. Also stores:

- doc_len: number of tokens per document
- df: document frequency per term (how many documents contain it)
- N: total number of documents in the corpus

Vocabulary size: 20,866 unique terms across the SciFact corpus.

- **retrieval_ranking.py - TF-IDF Baseline**

For each query, the system:

- Builds a TF-IDF weighted query vector
- Restricts candidate documents to those containing at least one query term (efficient pruning)
- Computes cosine similarity between the query vector and each candidate document vector
- Returns the top-100 ranked documents\

TF-IDF weighting formula used:

$$w(t, d) = (tf / doc\_len) * log2(N / df)$$

- **neural_rerank_mpnet.py - MPNet Reranker**

Reranks the TF-IDF top-100 using sentence-transformers/all-mpnet-base-v2:

- Encodes the query and each candidate document into 768-dimensional dense vectors
- Computes cosine similarity between the query embedding and each document embedding
- MPNet was fine-tuned on semantic similarity tasks, allowing it to capture meaning beyond keyword overlap

- **neural_rerank_USE.py - USE Reranker**

Reranks the TF-IDF top-100 using Google's Universal Sentence Encoder (v4):

- Produces 512-dimensional sentence embeddings
- Trained on a large variety of web text and conversational data

- Cosine similarity used for reranking

- evaluate.py

Computes standard IR evaluation metrics from TREC-format results files:

- MAP (Mean Average Precision): measures ranking quality across all queries
- P@10 (Precision at 10): measures how many of the top-10 results are relevant

4. Results

System	MAP	P@10
TF-IDF baseline (1st system)	0.5250	0.0778
TF-IDF baseline (2nd system)	0.5290	0.0833
MPNet reranker (best system)	0.6327	0.0940
USE reranker	0.3426	0.0560

Top 10 results - Query 1

Rank	Doc ID	Score	System
1	10607877	0.382897	MPNet
2	40212412	0.358548	MPNet
3	38037690	0.279652	MPNet
4	1065627	0.265662	MPNet
5	25404036	0.253317	MPNet
6	27049238	0.251082	MPNet
7	6863070	0.249835	MPNet
8	16736872	0.246487	MPNet
9	20758340	0.244026	MPNet
10	17123657	0.243300	MPNet

Top 10 results - Query 3

Rank	Doc ID	Score	System
1	23389795	0.646397	MPNet
2	14717500	0.642530	MPNet
3	1544804	0.640426	MPNet
4	1388704	0.639899	MPNet
5	2739854	0.625753	MPNet
6	13914198	0.621682	MPNet
7	15153602	0.621601	MPNet
8	32181055	0.589159	MPNet
9	4378885	0.585039	MPNet
10	2107238	0.581527	MPNet

5. Discussion

TF-IDF baseline

The TF-IDF baseline achieved a MAP of 0.5290, a reasonable result for a keyword-based system. The slight improvement over the first system's baseline (0.5250) comes from minor differences in preprocessing and normalization. The fundamental limitation of TF-IDF is vocabulary mismatch: if a query uses different words than the relevant documents, the system simply cannot make the connection. A query about "memory consolidation" may miss a document discussing "long-term potentiation" even if they describe the same phenomenon.

MPNet reranker

MPNet improved MAP by 19.6% over the TF-IDF baseline, from 0.5290 to 0.6327. This is a significant gain. The reason is straightforward: MPNet encodes sentences into a dense semantic space where similar meanings are close together, regardless of the specific words used. Two sentences can share zero vocabulary and still land near each other in embedding space if they mean the same thing. For scientific text, where vocabulary varies widely across authors and subfields, this matters a lot.

USE reranker

The Universal Sentence Encoder performed worse than even the TF-IDF baseline, with a MAP of 0.3426. This is a useful negative result. USE is a strong general-purpose encoder, but it was trained primarily on conversational and web data. Scientific language has very different statistical properties: dense terminology, passive voice, long nominal phrases. The model's embeddings do not represent this domain well, and the reranking ends up degrading the TF-IDF ordering rather than improving it.

This highlights a key insight that transfers directly to audio retrieval: not all embeddings are equal, and domain matters. A general-purpose audio encoder trained on YouTube videos would likely underperform a model specifically trained on music, just as USE underperforms MPNet on scientific text.

6. Connection to Music Information Retrieval

Building this system made the parallel to audio retrieval concrete. The same architectural decisions apply at every level:

Text IR	Audio IR (MIR)
Inverted index over terms	Inverted index over audio features or tags
TF-IDF query vector	Feature vector (MFCCs, chroma, tempo)
Neural sentence embeddings (MPNet)	Neural audio embeddings (openl3, CLAP)
Cosine similarity	Cosine similarity
MAP / P@10 evaluation	MAP / P@10 evaluation

The key question this project raised: if MPNet's semantic embeddings outperform TF-IDF on text, do learned audio embeddings outperform handcrafted features like MFCCs on music retrieval? This is exactly the question I explore in the playlist recommender project.

TO EXPAND

The most revealing result of this project is not that MPNet performed well, it is that USE failed. USE is a strong model, but it was trained on conversational and web text. Scientific language has very different statistical properties: dense terminology, passive voice, long nominal phrases. The model does not represent this domain well, and its reranking ends up degrading the TF-IDF ordering rather than improving it.

This failure mode has a direct parallel in audio retrieval. MFCCs, chroma, and spectral contrast were developed and validated primarily on Western music. They capture timbral and harmonic patterns well for the genres they were designed around, but they have no way to encode the polyrhythmic complexity of West African drumming, the specific low-end texture of Amapiano, or microtonal elements present in some vocal traditions. A retrieval system built on these features would behave exactly like USE on scientific text: it would find the nearest neighbor in a feature space that was never designed to represent what it is being asked about.

The USE vs MPNet comparison makes this concrete. The gap between them, MAP 0.3426 vs 0.6327; is not a gap in model size or architecture. It is a gap in training domain. MPNet was fine-tuned on semantic similarity tasks; USE was not. The domain of the training data determined the quality of the retrieval, more than anything else.

A natural extension of this project would be to apply the same pipeline to audio: replace the inverted index with an audio feature index, replace TF-IDF vectors with handcrafted audio features like MFCCs, and

replace MPNet with a neural audio encoder such as CLAP or openl3. Running the same MAP and P@10 evaluation would directly measure whether learned audio embeddings outperform handcrafted features on music retrieval, and whether the gap between them mirrors what was observed here between MPNet and USE.

7. How to Run

Requirements

```
pip install nltk sentence-transformers tensorflow tensorflow-hub
```

Run TF-IDF baseline

```
python runthesystem.py
```

Generates Results_baseline_tfidf.txt

Run neural rerankers

```
python neural_rerank_mpnet.py
python neural_rerank_USE.py
```

Generates Results_mpnet_rerank.txt and Results_use_rerank.txt

Evaluate

```
python evaluate.py Results
python evaluate.py Results_mpnet_rerank.txt
python evaluate.py Results_use_rerank.txt
```

The submitted Results file contains the MPNet reranker output, as it achieved the best overall performance.

8. References

- Reimers & Gurevych (2019): Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks
- Cer et al. (2018): Universal Sentence Encoder
- Thakur et al. (2021): BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models
- NLTK: stopwords and Porter Stemmer
- sentence-transformers/all-mpnet-base-v2 (HuggingFace)
- Universal Sentence Encoder v4 (TensorFlow Hub)